

Melyn take-home: Community Notices

Time budget: 2 to 3 hours. Due Thursday, Sep 24 by end of day. You may use any tools, including AI. Be ready to explain every part of what you submit.

Background

Melyn is a community platform for First Nations. Members use our mobile app to see notices from their community: emergency alerts, events, service announcements and general updates.

Build

A small React Native (TypeScript) app, bare or Expo, with two screens.

1. Notices list

- Newest first, 10 at a time, loading more as the user scrolls
- Pull to refresh
- Filter by category: all, emergency, event, service, general
- High priority notices stand out visually
- Read notices look different from unread
- Tapping a notice opens the detail screen

2. Notice detail

- Title, department, published date, body, and the image if there is one
- Opening a notice marks it as read. This must survive an app restart.

Data

Use the attached `notices.json` (45 notices). Do not call a real server. Build a small mock API layer inside the app that:

- reads from the JSON
- returns one page at a time (10 per page) with a short artificial delay
- fails roughly one request in five with an error, with a simple switch to turn failures off

The data includes a few real-world cases on purpose: a very long title, an empty body, a notice that has already expired, an image URL that will not load, and a published date in the future. Decide how the app should handle each and note your choice in the README.

What we look at

- Loading, empty and error states on both screens, including an error while loading more pages
- Poor or no network: show the last loaded notices, make it clear they may be stale, recover when a request succeeds

- A structure a teammate could extend: data access, state and UI kept separate
- Sensible handling of the edge cases in the data

Tests are not required, but say in the README what you would test first.

README (half a page is enough)

1. How to run it (iOS simulator or Android emulator)
2. Main decisions and trade-offs, including the edge cases
3. What you would add before shipping to real users: testing, crash reporting, releases, monitoring
4. What you would do differently with more time

Submit

A GitHub link (public, or private with access for [handle]) or a zip, plus a short screen recording or a few screenshots. On Friday we will walk through the code together and make a small change live, so have it ready to run.